

程序的机器级表示

Menghong Yu, EECS, PKU

2026/10/1

© SHANGHAI HENIAN INFORMATION TECHNOLOGY CO.,LTD

研讨题分享

BombLab配置

在您的终端键入如下指令(请将下面的`User`替换为您的用户名,例如`s`+学号):

```
mkdir -p /home/User/.config/gdb
vim /home/User/.config/gdb/gdbinit
```

然后在打开的vim编辑器中输入`i`进入编辑模式, 复制如下配置:

```
set auto-load safe-path /
set breakpoint pending on
set disassemble-next-line on
b explode_bomb
b phase_1
b phase_2
b phase_3
b phase_4
b phase_5
b phase_6
```

点击`esc`退出编辑模式, 输入`:wq`退出.

基本概念

basic concepts

- **Architecture（架构）**：也称为 ISA（指令集架构，Instruction Set Architecture）。

  指处理器设计的一部分，程序员需要了解，以便编写正确的汇编代码（asm Code）或机器码（Machine Code）。

  包括指令集规范（Instruction Set Specification）和寄存器（Registers）等。

- **Assembly Code（汇编代码）**

  是机器码的文本表示形式，通常使用助记符和符号来代替二进制代码，以便程序员更容易理解和编写。

- **Machine Code（机器码）**

  是处理器可以直接执行的字节级别程序。通常以二进制形式表示，是计算机硬件可以理解和执行的最低级别的指令。

CISC(复杂指令集计算机) vs RISC(精简指令集计算机)

需要会，很喜欢考

RISC

- 指令集精简，单条指令完成较简单的操作
- 指令编码规则，通常使用定长指令
- 寻址方式简单
- 通常采用 load/store 结构：算术指令主要操作寄存器，访存由专门指令完成
- 不使用条件码
- 使用寄存器传递过程参数

代表：ARM, MIPS, RISC-V

CISC

- 指令种类丰富
- 指令执行时间可能较长
- 编码长度可变
- 寻址方式多样：`Imm(%base, %index, scale)`
- 可以对内存操作数运算
- 通常有条件码
- 传统上常使用栈传递过程参数

代表：IA32, x86-64

发展趋势是融合发展。请参考课本 p249-250。

我们学习的 Y86-64 既有 CISC 特性也有 RISC 特性。

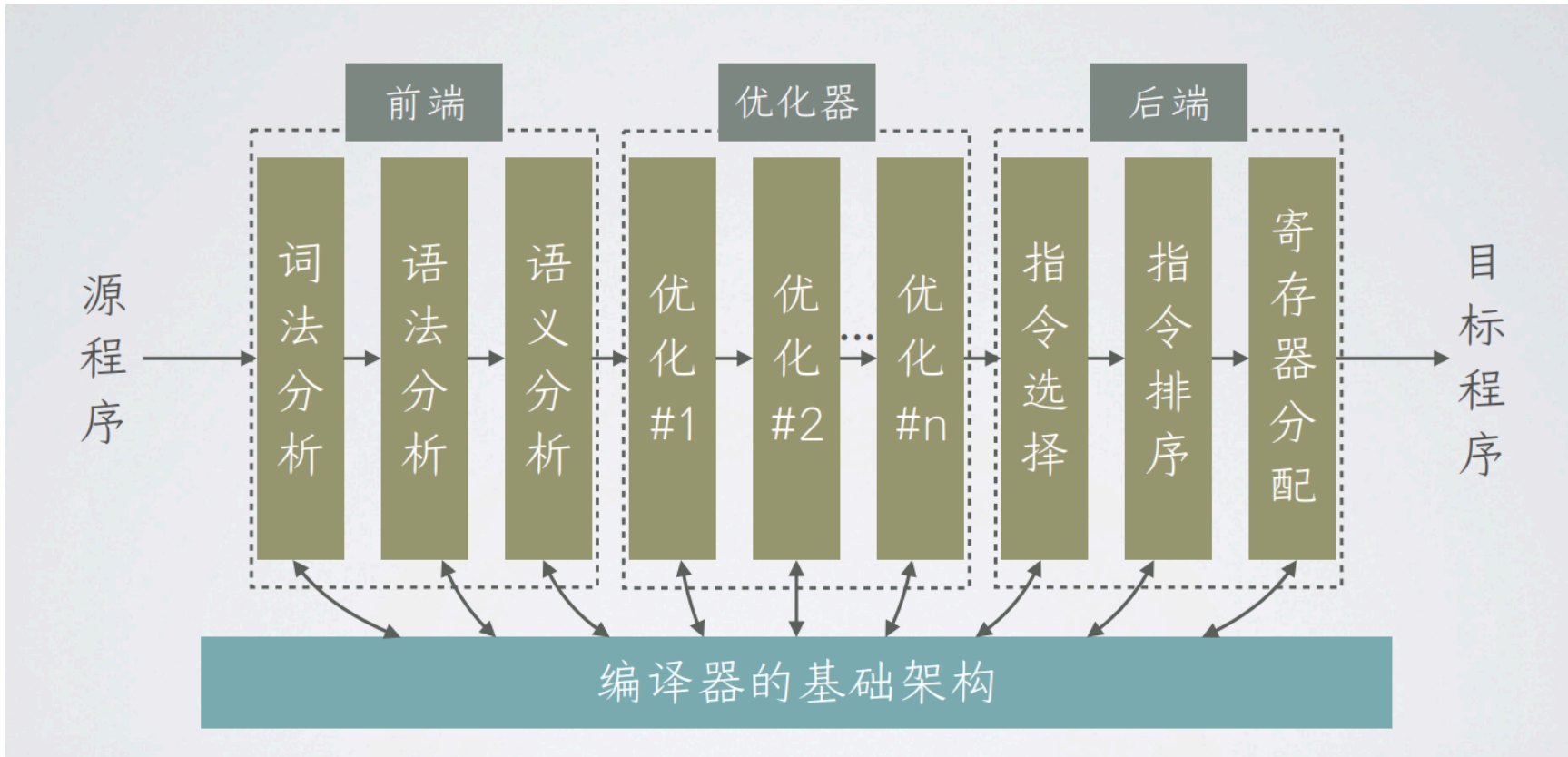
当你写完一段代码时，它如何起作用?

- 编译器工具链：将源代码逐步翻译成汇编代码和机器码，最终由 CPU 执行机器指令
  - 由于编译器在现代软件工程生态中的重要性，保证编译器的正确性成了一个较热门的研究方向
  - 感兴趣的同学可以阅读《CompCert - A Formally Verified Optimizing Compiler》
- 解释器：读取程序并按语言语义逐步执行，不一定先生成独立的机器码文件
- JIT编译：在运行期间把热点代码编译为机器码，以减少后续执行开销
  - 现代模型训练中有很多地方都使用了JIT编译优化，感兴趣的同学可以阅读《PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation》

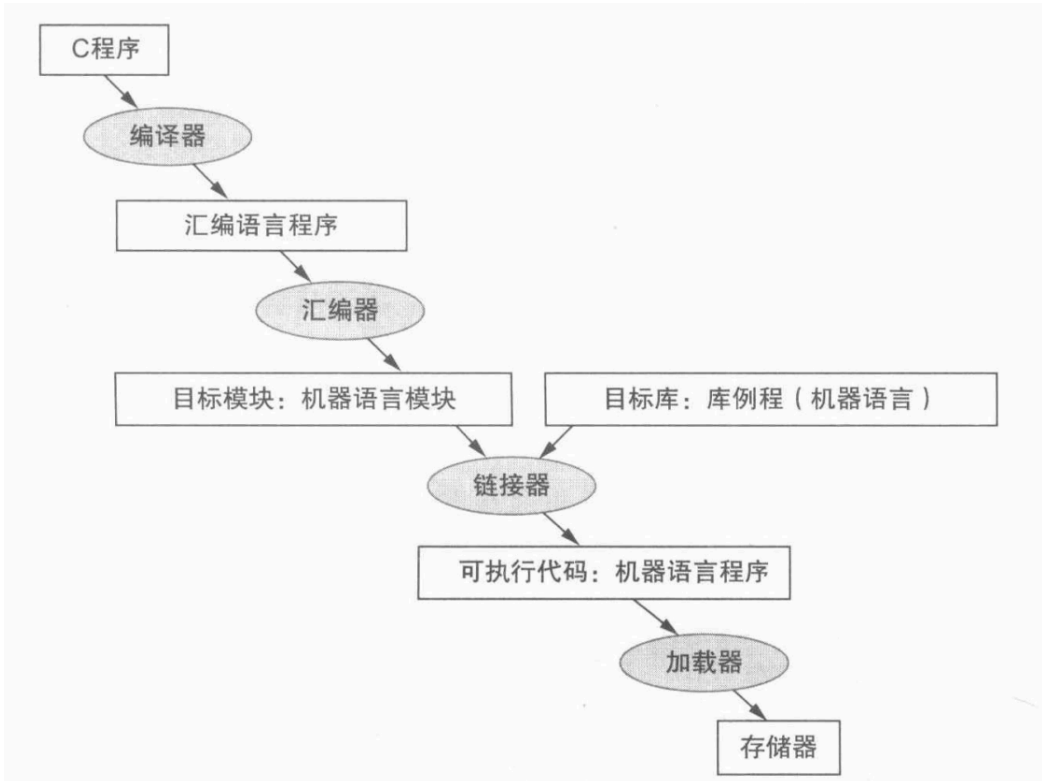
编译器是如何工作的?

- 源程序（source program）：通常使用一个编程语言来编写
- 目标程序（target program）：通常表示为一个处理器的指令集
- 有时会借助中间表示（IR, Intermediate Representation）

编译器是如何工作的?


前端
优化器
后端
源程序
词法分析
语法分析
语义分析
优化#1
优化#2
优化#n
指令选择
指令排序
寄存器分配
目标程序
编译器的基础架构

C语言编译器是如何工作的?



从 C 源代码到汇编代码

from C source code to assembly code

```
gcc -Og -S hello.c
gcc -Og -c hello.c
gcc -O1 -o hello hello.c
gcc -O1 -o hello hello.o
```

- 使用 `-S` 指令可以生成 `.s` 汇编代码文本
- 使用 `-c` 选项可以生成 `.o` 可重定位目标文件（只编译、汇编，不链接，在第七章“链接”中会讲）
- 使用 `-Og/-O1/-O2/-O3` 指令可以指定编译器优化等级

```
objdump -d hello > hello.asm
```

- 使用 `objdump` 可以将二进制文件通过反汇编得到它的汇编代码文本

基本概念

basic concepts

CPU 直接执行的是 **机器码**。汇编代码是机器指令的文本表示，汇编器会将其编码为目标文件中的机器指令。

指令的内容包括内存读写操作、算术操作、控制操作等。

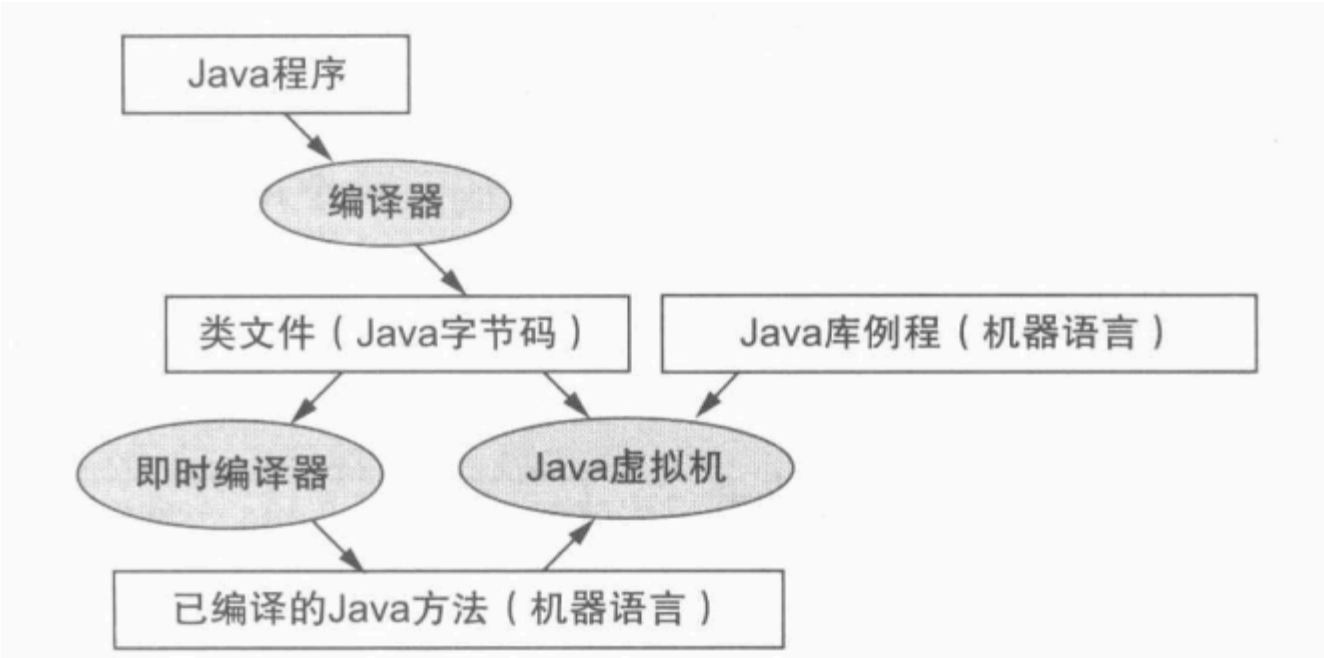
使用 Objdump 反编译一个二进制文件（在 Bomblab 你一定会用到）：

```
objdump -d bomb > bomb.asm
```

- `-d` 代表 disassemble，反汇编。
- `bomb` 是你要反汇编的二进制文件名。
- `> bomb.asm` 是将反汇编结果输出到 `bomb.asm` 文件中。

```
0000000000001000 <_init>:
    1000: f3 0f 1e fa                   endbr64
    1004: 48 83 ec 08                   subq    $8, %
    1008: 48 8b 05 d9 5f 00 00          movq    24537
    100f: 48 85 c0                      testq   %rax,
    1012: 74 02                         je  0x1016 <_
    1014: ff d0                         callq   *%rax
    1016: 48 83 c4 08                   addq    $8, %
    101a: c3                            retq
```

JAVA语言编译器是如何工作的?



数据格式

data format

C 声明	Intel 数据类型	汇编代码后缀 (记忆)	大小（字节）
`char`	字节	b (byte)	1
`short`	字	w (word)	2
`int`	双字	l (double word)	4
`long`	四字	q (quad word)	8
`char*`	四字	q (quad word)	8
`float`	单精度	s (single word)	4
`double`	双精度	l (double word)	8

记忆：byte, word, long word, quad word

程序计数器

Program Counter (PC)

- 程序计数器（PC）是一个寄存器，用于存储将要执行的**下一条指令**在内存中的地址
  - 为什么需要PC? 因为冯诺依曼架构: 存储程序原理
- 在 x86-64 中，程序计数器通常是 `%rip` 寄存器。
- 每当执行一条指令时，程序计数器会根据指令的长度自动更新，指向下一条指令的地址。

存储

storage

程序在运行时的存储主要有 **寄存器（条件码）** 和 **内存**。

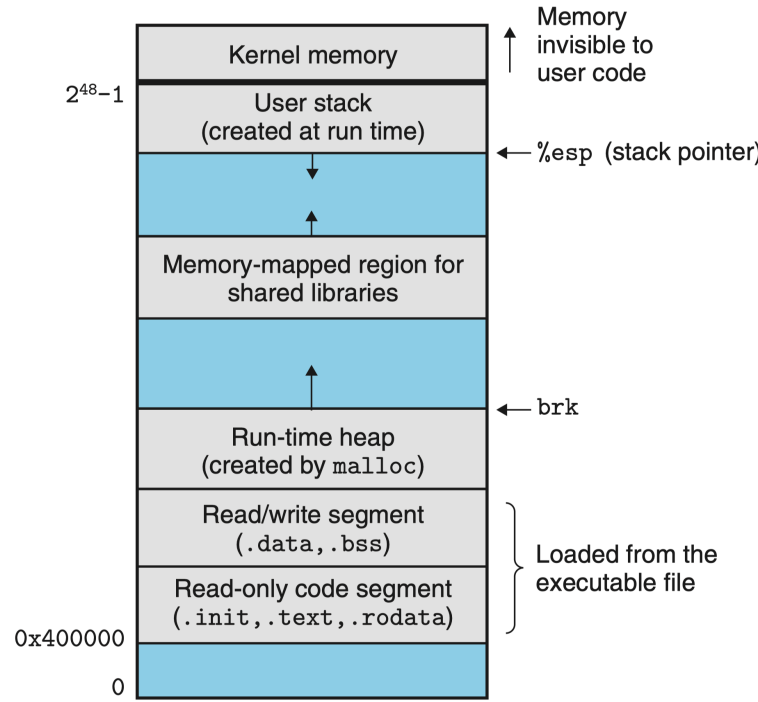
- 内存可以抽象为按字节寻址的数组，地址宽度决定了可表示的地址范围。
- 通用寄存器可以看作运行过程中可快速访问的存储位置；部分寄存器具有硬件规定的用途，调用约定还会赋予通用寄存器特定角色。

栈

stack

栈在程序运行时具有很重要的作用，包括：

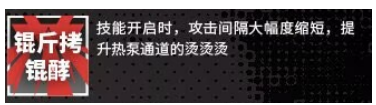
- **函数调用管理**：函数调用使用栈保存返回地址，并在需要时建立栈帧。
- **局部变量存储**：无法完全保存在寄存器中的局部数据可以放入栈帧；函数结束时通常通过恢复 `%rsp` 释放这部分空间。
- **控制流管理**：通过保存返回地址，确保函数调用结束后能正确返回调用点。

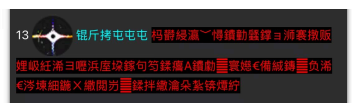


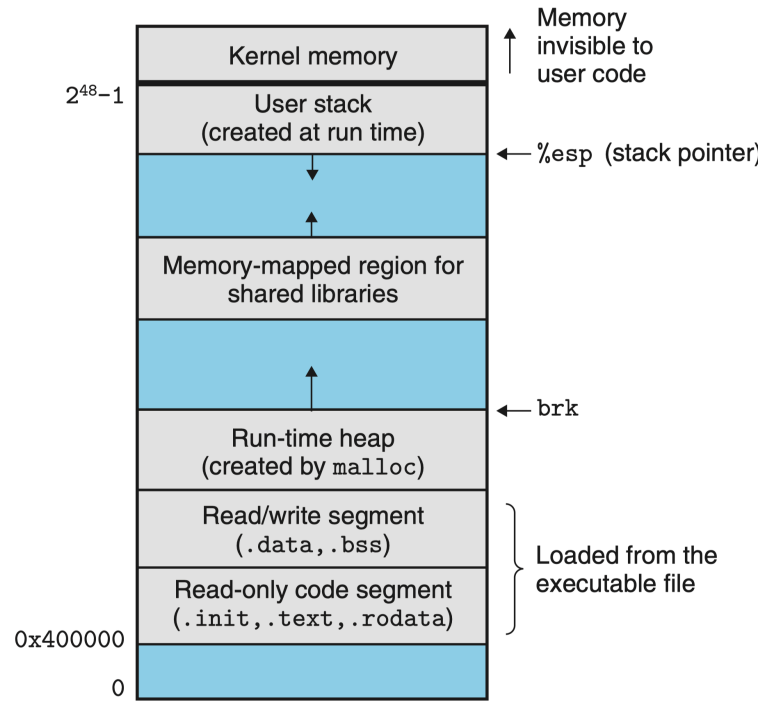
程序输出烫烫烫烫烫烫，是不是CPU过热?

hot!hot!hot!!!

- MSVC调试版中，栈中的未用字节初始化为 `0xCC`，而"烫"字的GBK编码为 `0xCCCC`.
- 同理，堆中的未用字节初始化为 `0xCD`，而"屯"字的GBK编码为 `0xCDCD`.







各种乱码一览

Mojibake

- 常用中文编码有UTF-8和GBK两种, 其中UTF-8通常以三字节编码一汉字, GBK以二字节编码一汉字
  - 末尾问号: UTF-8 编码的汉字总字节数如果是奇数n，则总共使用了3n个字节，此时用 GBK 强行读取时，最后一个孤零零的字节由于凑不够一对，就会被识别为非法字节，从而被永久破坏成了问号。
  - `�`: UTF-8中对识别不出的字符会用`�`(编码为 `0xEF 0xBF 0xBD`)代替
  - 锘斤拷: 如果此时再转为GBK编码(二字节编码汉字), "锘斤拷"三个字的编码分别是 `0xEF 0xBF` , `0xBD 0xEF` , `0xBF 0xBD` .

名称	示例	特点	产生原因
古文码	墨研溴瑕恬ソ濂藉滓财ぉ谹十惺涓?	大都为不认识的古文，并加杂日韩文	以GBK方式读取UTF-8编码的中文
口字码	����Ç�¯2�ax������	大部分字符为小方块	以UTF-8的方式读取GBK编码的中文
符号码	ç"±æœ¤è|□å¥½å¥½ï½å-|ã1 å¤©å¤©å□ˆä¸Š	大部分字符为各种符号	以ISO8859-1方式读取UTF-8编码的中文
拼音码	óéÔÅòaoÃoÃÑ§Ï°ìiiÏòéÏ	大部分字符为头顶带有各种类似声调符号的字母	以ISO8859-1方式读取GBK编码的中文
问句码	由月要好好学习天天向??	字符串长度为偶数时正确，长度为奇数时最后的字符变为问号	以GBK方式读取UTF-8编码的中文，然后又用UTF-8的格式再次读取
锘拷码	锘斤拷锘斤拷要锘娇娘拷学习锘斤拷锘斤拷锘斤拷	全中文字符，且大部分字符为"锘斤拷"这几个字符	以UTF-8方式读取GBK编码的中文，然后又用GBK的格式再次读取

寄存器

Registers

这个表要完整的记忆下来。



寄存器

Registers

- `%r[a-d]x` - `%e[a-d]x` - `%[a-d]x` - `%[a-d]l`

  Register / Extended / Lower-part

  - accumulator 累加寄存器 / **返回值**
  - base 基址寄存器
  - count 计数寄存器
  - data 数据寄存器

  记忆：只有 `[a-d]x` 的 Low-part 除了尾加 `l` 以外，还删掉了原有的 `x` 后缀

寄存器

Registers

- `%r[s/d]i` - `%e[s/d]i` - `%[s/d]i` - `%[s/d]il`

  Register / Extended / Lower-part

  - source index 源索引寄存器
  - destination index 目的索引寄存器

  记忆：此处恢复正常，在 `[s/d]i` 后面直接加代表 low 的 `l` 后缀

寄存器

Registers

- `%r[b/s]p` - `%e[b/s]p` - `%[b/s]p` - `%[b/s]pl`

  Register / Extended / Lower-part

  - base pointer 栈基址寄存器
  - stack pointer 栈指针寄存器

  记忆：此处恢复正常，在 `[b/s]p` 后面直接加代表 low 的 `l` 后缀

寄存器

Registers

- `%r[8~15]` - `%r[8~15]d` - `%r[8~15]w` - `%r[8~15]b`

  Register / Double-word / Word / Byte

  - 命名非常规则

寄存器

Registers

被调用者保存寄存器

- `%rbx`
- `%rbp`
- `%r12` - `%r15`

调用者保存寄存器

除了上述寄存器以及 `%rsp` 外，其余通用寄存器均为调用者保存寄存器。

什么是被调用者保存寄存器?

- 当一个函数 `B` 被另一个函数 `A` 调用时，被调用者 `B` 在处理过程中，有可能覆盖一些寄存器，然而这些寄存器对于恢复 `A` 的执行状态非常重要。
- 因此，在 `B` 的开始处，将这些寄存器中的值 **保存** 到栈中，在 `B` 的结束处，再将这些寄存器中的值从栈中 **恢复**。
- 所以，我们称这些寄存器为被调用者（`B`）保存寄存器。

寄存器

Registers

参数寄存器

- `%rdi`：第一个参数
- `%rsi`：第二个参数
- `%rdx`：第三个参数
- `%rcx`：第四个参数
- `%r8`：第五个参数
- `%r9`：第六个参数

硬记吧，反正就 6 个。

其他参数

超过 6 个参数的函数调用，**使用栈传递**。

操作数指示符

operand indicator

操作数主要格式如下图。注意：

- 在 x86-64 中，基址和变址寄存器必须是 64 位寄存器（记忆：64 位系统）→ `(%eax)` 不合法
- 比例因子必须是 1、2、4、8（记忆：二进制）→ `(,%rax,3)` 不合法
- **特殊记忆：%rsp 不能作为变址寄存器**，即 `Imm(%base, %index, scale)` 中的 %index。

Mov 类指令

`mov` instruction

MOV S, D	S→D	传送
`movb`		传送字节
`movw`		传送字
`movl`		传送双字 *
`movq`		传送四字
`movabsq I, R`	I→R	传送绝对的四字

- D 表示 destination / 目的地，S 表示 source / 源。二者不能同时为内存
- 寄存器大小必须和指令后缀匹配。
- * `movl` 指令以寄存器为目的时，会将高位 4 字节清零。
- `movq` 只能处理 32 位立即数的源操作数，如果需要处理 64 位立即数只能使用 `movabsq`（想想为什么？）

Mov 类指令

`mov` instruction

S, R	`movz`	`movs`
1→2	`movzbw`	`movsbw`
1→4	`movzbl`	`movsbl`
1→8	`movzbq`	`movsbq`
2→4	`movzwl`	`movswl`
2→8	`movzwq`	`movswq`
4→8	`movl`	`movslq`
4→8	/	`cltq`

`cltq`：convert long to quad，将 `%eax` 符号扩展到 `%rax`，等价于 `movslq %eax, %rax`。

`s`：代表符号扩展，`z`：代表零扩展。

想想为什么没有 `movzlq`？

Mov 类指令

`mov` instruction

在 x86-64 下，以下哪个选项的说法是错误的?

- A. `movl` 指令以寄存器为目的时，会将该寄存器的高位 4 字节设置为 0
- B. `cltq` 指令的作用是将 `%eax` 符号扩展到 `%rax`
- C. `movabsq` 指令只能以寄存器为目的地
- D. `movswq` 指令的作用是将零扩展的字传送到四字节目的地

Mov 类指令

`mov` instruction

x86-64 汇编	错误原因
`movb $0xF, (%ebx)`	基址寄存器应为 64 位
`movl %rax, (%rsp)`	应为 `movq`
`movw (%rax), 4(%rsp)`	mov 类不能 M -> M
`movb %al, %sl`	没有 `%sl`
`movq %rax, $0x123`	反了
`movl %eax, %rdx`	`movl` 和 `%rdx` 矛盾
`movb %si, 8(%rbp)`	`movb` 应对应 `%sil`

算数和逻辑操作

arithmetic and logical operations

指令	效果	描述
`LEAQ S D`	D ← &S	加载有效地址
`INC D`	D ← D + 1	加 1
`DEC D`	D ← D - 1	减 1
`NEG D`	D ← -D	取负
`NOT D`	D ← ~D	取反（按位）
`ADD S D`	D ← D + S	加法
`SUB S D`	D ← D - S	减法
`IMUL S D`	D ← D * S	乘法

指令	效果	描述
`XOR S D`	D ← D ^ S	按位异或
`OR S D`	D ← D | S	按位或
`AND S D`	D ← D & S	按位与
`SAL k D`	D ← D << k	左移
`SHL k D`	D ← D << k	左移（与 SAL 相同）
`SAR k D`	D ← D >>$_A$ k	算术右移
`SHR k D`	D ← D >>$_L$ k	逻辑右移

leaq 指令

leaq instruction

`leaq` 指令：load effective address，意为“加载有效地址”。**但实际上主要用于计算**

- “加载”是取内存的意思，地址代表取值，相当于 C 中先 `*` 再 `&`，等于没与内存交互。

  所以 `leaq` 指令主要用于计算，比如 `leaq 7(%rdx,%rdx,4), %rax` 意为 `%rax = 7 + 5 * %rdx`

- `leaq` 支持多种寻址方法，但是要求 `D` 必须是寄存器
- `leaq` 不改变条件码（常考）

暗坑之省略也要加逗号：

```
1    leaq (%rdx, 1), %rdx
```

应当改为：

```
1    leaq (,%rdx,1), %rdx
```

移位操作

shift operation

移位量可以是一个立即数，或者放在单字节寄存器 `%cl` 中。（为什么是放在单字节寄存器中呢？）

注意：只有这个寄存器可以存移位量！（记忆：`cl` → count lower）

左移（`SAL` / `SHL`）是等价的，但是右移会区分算术右移（`SAR`，前补符号位）和逻辑右移（`SHR`，前补 0）。

特殊算数操作

special arithmetic operations

指令	效果	描述
`imulq S`	`R[%rdx]:R[%rax] ← S × R[%rax]`	有符号全乘法
`mulq S`	`R[%rdx]:R[%rax] ← S × R[%rax]`	无符号全乘法
`cqto`	`R[%rdx]:R[%rax] ← SignExtend(R[%rax])`	将 `%rax` 符号扩展为 128 位被除数
`idivq S`	`R[%rax] ← dividend ÷ S; R[%rdx] ← dividend mod S`	有符号除法；`dividend` 为原 `%rdx:%rax`
`divq S`	`R[%rax] ← dividend ÷ S; R[%rdx] ← dividend mod S`	无符号除法；`dividend` 为原 `%rdx:%rax`

一般这里只考搭配的是 `%rax` 和 `%rdx` 两个寄存器，且后者在乘法里是高位。`%rax` 在除法里用于存商（因为 `%rax` 一般用于返回结果，可以这么记）

条件码

condition code / flags

算术和逻辑指令执行后，通常会隐式修改以下 4 个**条件码**：

- ZF（Zero Flag）：当该次运算结果为 0 时被置为 1，否则置为 0
- SF（Sign Flag）：当运算结果的符号位（最高位）为 1 时被置为 1，否则置为 0
- CF（Carry Flag）：当两个 unsigned 类型的数作运算因 **进位/借位** 而发生溢出时置为 1，否则置为 0
- OF（Overflow Flag）：当两个 signed 类型的数做运算而发生符号位溢出时置为 1，否则置为 0

进位和溢出的区分

CF and OF

- 对二进制加法（含用二补码表示的有符号加法）：
  - CF = 最高位之后产生的进位（carry-out of MSB）。
  - OF = “进位进入最高位” XOR “进位从最高位溢出”。 等价的直观判定：如果两个被加数 符号位相同，且和的符号与它们不同，则 OF = 1。
- 对减法（`x - y`，注意计算条件码时不等价于 `x + (~y + 1)`）：
  - CF = 1 表示发生了借位（等价于 unsigned 下 x < y）。
  - OF 的判定也可用符号法：如果被减数和减数 符号不同，且结果的符号与被减数不同，则 OF = 1。

CF 和 OF：小练习

Exercise

下面考虑 8 位整数。

1. `0x7F + 0x01`
   - `0x80` (128 as unsigned, -128 as signed)
   - `CF = 0`
   - `OF = 1`
2. `0x00 - 0x80`
   - `0x80`
   - `CF = 1`
   - `OF = 1`（`0 - (-128) = 128`，符号溢出）
3. `0x00 - 0x01`
   - `0xFF`
   - `CF = 1`（无符号减法发生借位）
   - `OF = 0`（`0 - 1 = -1`，没有符号溢出）
4. `0x80 - 0x01`
   - `0x7F`
   - `CF = 0`（无符号 `128 - 1 = 127`）
   - `OF = 1`（`-128 - 1 = -129`，溢出）

条件码

condition code / flags

设置条件码的细节：

- `leaq` 指令不改变任何条件码
- 逻辑操作（`AND`、`OR`、`XOR`）会把 `CF` 和 `OF` 设置成 0，`NOT` 不影响任何标志码
- 移位量非零时，移位操作会把 `CF` 设置为最后一个被移出的位
- 右移 1 位时，`SAR` 将 `OF` 清零，`SHR` 将 `OF` 设置为原最高位；移位量大于 1 时 `OF` 未定义
- 左移 1 位时，`OF = MSB(result) XOR CF`；移位量大于 1 时 `OF` 未定义
- `INC` 和 `DEC` 会更新 `OF`、`SF`、`ZF`、`AF` 和 `PF`，但不会改变 `CF`

条件码指令

`cmpq` and `testq`

`cmpq s1, s2`

包括 `cmpb` / `cmpw` / `cmpl` / `cmpq`

计算 `s2 - s1`，但不同于 `subq` 会修改 `s2` 的值，`cmpq` 只会影响条件码的值。

注意顺序！记忆：大多数指令都是源操作数在前，目的操作数在后

`testq s1, s2`

包括 `testb` / `testw` / `testl` / `testq`

计算 `s1 & s2`，同样只会影响条件码的值。

条件码指令

`setX`

我们并不能直接访问 Condition Codes，但我们可以用指令 `setX` 来间接的获取它们的值。

setX	操作	描述
`sete D`	`D ← ZF`	等于 0 时设置
`setne D`	`D ← ~ZF`	不等于 0 时设置
`sets D`	`D ← SF`	负数时设置
`setns D`	`D ← ~SF`	非负数时设置

需要注意的是，`setX` 只会将最低的 1 个 Byte 置为相应的值，其他 Byte 保持不变！

Q：如果想要设置全部 8 Byte，该使用什么指令？ A：`movzbl` / `movzbq`

条件码指令

`setX`

setX	操作	描述
`setg D`	`D ← ~(SF^OF)&~ZF`	大于（有符号）
`setge D`	`D ← ~(SF^OF)`	大于或等于（有符号）
`setl D`	`D ← (SF^OF)`	小于（有符号）
`setle D`	`D ← (SF^OF)|ZF`	小于或等于（有符号）
`seta D`	`D ← ~CF&~ZF`	大于（无符号）
`setb D`	`D ← CF`	小于（无符号）

小于条件的证明

proof of `setl`

`cmp b a` 当 $a < b$ 时，有两种情况：

1. **有溢出的情况**：
   - 条件： $a < b$ 且 $(a-b)_{\text{补码}} > 0$
   - 结果：会发生负溢出
   - 标志位： $OF = 1, SF = 0$
2. **无溢出的情况**：
   - 条件： $a < b$ 且 $(a-b)_{\text{补码}} < 0$
   - 结果：不会发生溢出
   - 标志位： $OF = 0, SF = 1$（负数）

由此，判断是否小于，可以用异或表达式 `OF ^ SF` 来表示。

而且，我们无法直接获得 `OF` 的值。

跳转指令

jumping instruction

直接跳转

```
1   Label:
2     ...
3     jmp Label
```

跳转目标作为指令的一部分编码，要定义 `Label`

`Label` 是汇编符号；汇编器和链接器会把它解析为相对位移或相应的重定位信息

间接跳转

```
1   jmp *Operand    e.g. jmp *%rax ; jmp *(%rax)
```

跳转目标是从寄存器或内存位置中读出的，有一个类似先前 `mov` 指令中介绍的取值表达式

在后面介绍的 `switch` 跳转表中会用到！

跳转指令的编码

encoding of jumping instruction

PC 相对的（PC-relative）

将目标指令的地址与紧跟在跳转指令后面那条指令的地址之间的差作为编码

- 当执行 `PC` 相对寻址时，**程序计数器的值是跳转指令后面的那条指令的地址**
- 在 x86-64 的常见近跳转编码中，地址偏移量为 **1 或 4** 个字节，按有符号数解释

好处：

- 指令编码更简洁：短跳转使用 1 字节位移，常见近跳转使用 4 字节位移；整条指令还包括操作码等字段

间接指定绝对目标

x86-64 的近直接跳转通常编码为相对位移。若要跳转到寄存器或内存中保存的任意绝对地址，应使用 `jmp *Operand` 形式的间接跳转。

PC 相对的跳转指令的编码

encoding of PC-relative jumping instruction

注意，其中一个加数的是 下一条指令 的地址!

```
1     movq    %rdi, %rax
2     jmp     .L2
3   .L3:
4     sarq    %rax
5   .L2:
6     testq   %rax, %rax
7     jg      .L3
8     rep; ret
```

The disassembled version of the `.o` format generated by the assembler is as follows:

```
1     0:   48 89 f8              mov    %rdi,%rax
2     3:   eb 03                 jmp    8 <loop+0x8>
3     5:   48 d1 f8              sar    %rax
4     8:   48 85 c0              test   %rax,%rax
5     b:   7f f8                 jg     5 <loop+0x5>
6     d:   f3 c3                 repz retq
```

PC 相对的跳转指令的编码

encoding of PC-relative jumping instruction

在如下代码段的跳转指令中，目的地址是?

```
1   400020: 74 F0     je ____
2   400022: 5d        pop %rbp
```

- `je` 指令：1 字节操作码 + 1 字节偏移量
- 偏移量范围：-128 到 +127（共256个字节）

`0xF0` 是一个负数，我们使用 `-x = ~x + 1` 来快速准确转换

`~0xF0 = 0x0F, 0x0F + 1 = 0x10`

`0x400022 - 0x10 = 0x400012`

条件跳转指令

conditional branches & jumping

以 `.Expr:` 的格式定义跳代码块后，可以利用 `jX` 指令跳转到指定的代码位置。

利用不同的 jumping 指令（定义类似之前的 `setX` 指令），我们可以实现条件分支。

jX	条件
`jmp`	`1`
`je`	`ZF`
`jne`	`~ZF`
`js`	`SF`
`jns`	`~SF`

jX	Condition
`jg`	`~(SF^OF)&~ZF`
`jge`	`~(SF^OF)`
`jl`	`(SF^OF)`
`jle`	`(SF^OF)|ZF`
`ja`	`~CF&~ZF`
`jb`	`CF`

条件跳转指令示例

conditional branches example

```c
long absdiff(long x, long y) {
    long result;
    if(x > y) result = x-y;
    else result = y-x;
    return result;
}
```

```asm
absdiff:
    cmpq %rsi, %rdi  # x:y
    jle .L4
    movq %rdi, %rax
    subq %rsi, %rax
    ret
.L4:        # x <= y
    movq %rsi, %rax
    subq %rdi, %rax
    ret
```

等价的 `goto` 版本：

```c
long absdiff_goto(long x, long y){
    long result;
    if(x > y) goto x_gt_y;
    result = y-x;
    return result;
x_gt_y:
    result = x-y;
    return result;
}
```

注意这里对 `if else` 的转写，在 Archlab 中会用到！

条件传送指令

`cmov` (conditional move) instruction

`cmovX S, R`：当条件码 `X` 满足时，将源寄存器或内存操作数的值 `S` 传送到目的寄存器 `R`

- **不支持单字节的条件传送**
- 汇编器可以从目标寄存器的名字推断出条件传送指令的操作数长度，而无需显式写出（Diff：`movw`、`movl`、`movq`）
- 与流水线预测有关（第四章会学），并不总会提升效率

条件传送的适用条件

conditional move applicable conditions

只有当两个表达式（then-expr 和 else-expr）都很容易计算时，GCC才会使用条件传送

一些不会使用条件传送的情况：

1. 表达式的求值需要大量的运算
2. 表达式的求值可能导致错误

```
1    val = p ? *p : 0;
```

在这个式子中，涉及指针的解引用操作，有可能会导致错误，如 `p` 为空指针

3. 表达式的求值可能导致副作用

```
1    val = x > 0 ? x *= 7 : x += 3;
```

这两个操作都会修改 `x` 的值，称为副作用。条件传送要求在硬件层面一次性完成选择操作，而不能处理带有副作用的表达式。

如果使用条件传送，无法保证 `x` 的修改只发生在条件满足的情况下。

条件传送的适用条件

conditional move applicable conditions

对于下列四个函数，假设 gcc 开了编译优化，判断 gcc 是否会将其编译为条件传送

```
long f1(long a, long b) {
    return (++a > --b) ? a : b;
}
```

```
long f2(long *a, long *b) {
    return (*a > *b) ? --(*a) : (*b)--;
}
```

```
long f3(long *a, long *b) {
    return a ? *a : (b ? *b : 0);
}
```

```
long f4(long a, long b) {
    return (a > b) ? a++ : ++b;
}
```

你可以使用 在线编译器 来验证！（注意别在你的 ARM 设备上试，会不一样）

- `f1` 由于比较前计算出的 `a` 与 `b` 就是条件传送的目标，因此会被编译成条件传送；
- `f2` 会修改被选中指针指向的元素，因此会被编译成条件跳转；
- `f3` 由于指针 `a` 可能无效，因此会被编译为条件跳转；
- `f4` 会被编译成条件传送，注意到 `a` 和 `b` 都是局部变量，return 的时候对 `a` 和 `b` 的操作都是没有用的。

do-while 循环

do-while loop

```
do
    body-statement
while (test-expr);
```

```
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
```

此语法结构决定了循环体至少执行一次。

```
long fact_do(long n) {
    long result = 1;
    do {
        result *= n;
        n = n - 1;
    } while (n > 1);
    return result;
}
```

```
; long fact_do(long n)
; n 存储在 %rdi
fact_do:
    movl $1, %eax       ; 初始化 result = 1
.L2:
    imulq %rdi, %rax    ; 计算 result *= n
    subq $1, %rdi       ; 计算 n = n - 1
    cmpq $1, %rdi       ; 比较 n : 1
    jg .L2              ; 如果 n > 1，跳转到 .L2
    rep; ret            ; 返回
```

while 循环

while loop

```
while (test-expr)
    body-statement
```

```
goto test;
loop:
    body-statement
test:
    t = test-expr;
    if (t)
        goto loop;
```

```
long fact_while(long n) {
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n - 1;
    }
    return result;
}
```

```
; long fact_while(long n)
; n 存储在 %rdi
fact_while:
    movl $1, %eax       ; 初始化 result = 1
    jmp .L5             ; Goto test
.L6:
    imulq %rdi, %rax    ; 计算 result *= n
    subq $1, %rdi       ; 计算 n = n - 1
.L5:
    cmpq $1, %rdi       ; 比较 n : 1
    jg .L6              ; 如果 n > 1, 跳转到 .L6
    rep; ret            ; 返回
```

while 循环 - guarded do

while loop - guarded do

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

```
long fact_while(long n) {
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n - 1;
    }
    return result;
}
```

```
; long fact_while(long n)
; n 存储在 %rdi
fact_while:
    cmpq $1, %rdi       ; 比较 n : 1
    jle .L7             ; 如果 <=, 跳转到 done
    movl $1, %eax

.L6:
    imulq %rdi, %rax    ; 计算 result *= n
    subq $1, %rdi       ; 计算 n = n - 1
    cmpq $1, %rdi       ; 比较 n : 1
    jne .L6             ; 如果 n != 1, 跳转到 .L6
    rep; ret            ; 返回
.L7:
    movl $1, %eax       ; 计算 result = 1
    ret                 ; 返回
```

for 循环

for loop

```
for (init-expr; test-expr; update-expr)
    body-statement
```

只需转换为 while 循环：

```
init-expr;
while (test-expr) {
    body-statement
    update-expr;
}
```

Switch 语句

switch statement

使用跳转表的场景：

- 当开关情况数量比较多（例如 4 个以上）
- 值的范围跨度比较小时（数据的极差不能过大）

优点：执行开关语句的时间与开关情况的数量无关。可以看做一种用空间换时间的策略

具体实现：

- 在 `.rodata`（只读数据）段中，用连续的地址存放一张跳转表，跳转表中的每一个表项对应了 switch 的一个分支代码段的所在地址
- 在 `switch` 对应的汇编代码段中，用 `jmp *JumpTab[x]` 间接跳转的方式跳转到对应的分支

简便理解：在一段特定的内存空间（跳转表）中，连续存放了多个跳转指令起始入口的地址，然后通过间接跳转的方法，根据当前的 `case` 值，跳转到对应的 `case` 代码段中。

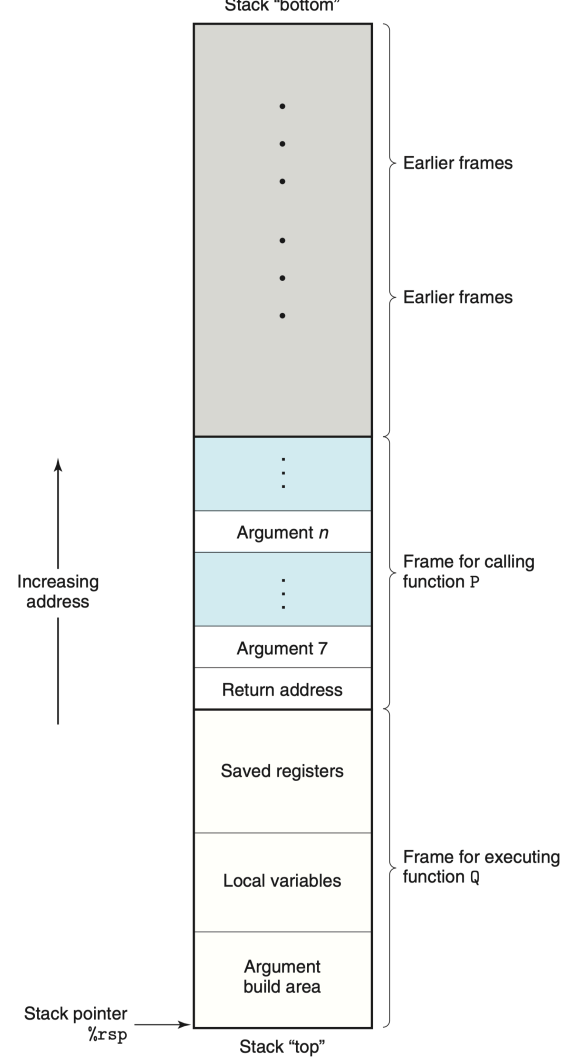
栈

stack

- 栈从高地址向低地址增长 （向下增长）
- `%rsp` 表示栈中最低地址元素所在的位置（栈顶）
- `%rbp` 表示当前栈帧中一个锚点
- 超过 6 个参数的函数调用，使用栈传递参数

回忆一下，第 1~6 个参数分别使用哪些寄存器传递？

`%rdi`，`%rsi`，`%rdx`，`%rcx`，`%r8`，`%r9`



栈的数据操作

stack data operation

指令	影响	描述
`pushq S`	`R[%rsp] ← R[%rsp] - 8; M[R[%rsp]] ← S`	压入四字（quad）数据
`popq D`	`D ← M[R[%rsp]]; R[%rsp] ← R[%rsp] + 8`	弹出四字（quad）数据

注意顺序！（其实这个顺序也很好理解，`stk[++top] = a` 和 `a = stk[top--]` 罢了）

但是需要注意的是，**对于操作数是 `%rsp` 的时候**，问题就不是那么简单了。

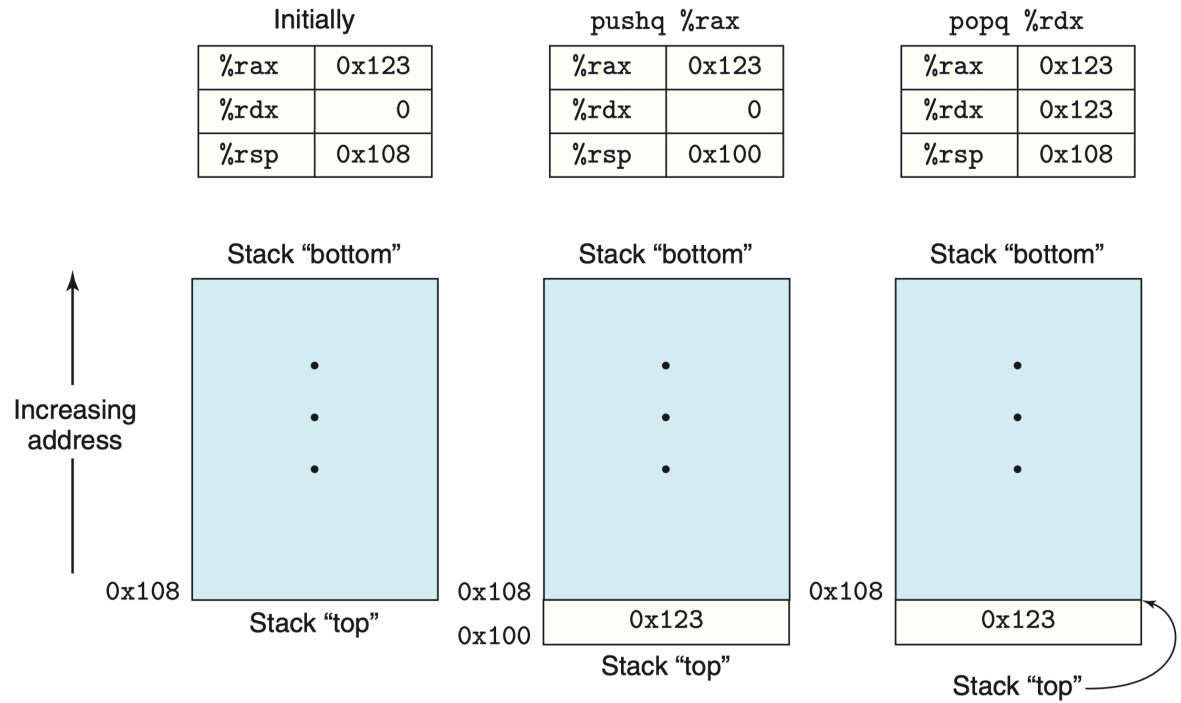
思考一下 `pushq %rsp` 和 `popq %rsp` 的时候会发生什么

- pushq %rsp 压的是 %rsp 的旧值；
- popq %rsp 是将栈指针设置为栈顶读出来的值

可以参考课本 p255-256 练习题 4.7 和 4.8, 我们讲Arch的时候还会细讲

栈的数据操作

stack data operation



过程

procedure

过程是软件设计中的重要抽象概念，它提供了一种封装代码的方式。通过指定的参数和可选的返回值来实现某个特定功能。

深入到机器层面，过程基于如下机制：

- **传递控制**：在进入过程 Q 的时候，程序计数器必须被设置为 Q 的代码的起始地址，然后在返回时，要把程序计数器设置为 **P 中调用 Q 后面那条指令** 的地址。
- **传递数据**：P 必须能够向 Q 提供一个或多个参数，Q 必须能够向 P 返回一个值。
- **分配和释放内存**：在开始时，Q 可能需要为局部变量分配空间，而在返回前，又必须释放这些存储空间。

运行时栈

runtime stack

C 语言过程调用依赖于运行时栈进行数据和指令管理。

- **过程栈帧**：函数在需要栈空间时会分配栈帧，用于保存局部数据、部分参数和寄存器等；函数调用还会在栈上保存返回地址。
- **栈帧管理**：函数可通过调整 `%rsp`，并按需使用 `push` / `pop` 或普通访存指令来管理栈上数据。

  栈帧不是必要的；寄存器不足、局部变量取地址或需要保存寄存器时会使用栈空间。
- **栈顶与栈底**：通过调整栈顶指针 `%rsp` 和栈底指针 `%rbp` 来管理栈帧。

  注意，栈顶在低地址，栈底在高地址，栈是向下增长的。

x86-64 的栈结构

stack structure in x86-64

1. **栈帧布局**：
   - **参数构造区**：存放函数构造的参数

     看图的上面，参数 7~n 就是 P 的参数构造区，注意顺序
   - **局部变量区**：用于函数临时构造的局部变量。
   - **被保存的寄存器**：用于保存调用过程中使用到的寄存器状态

     被调用者保存寄存器： `%rbp` `%rbx` `%r12` `%r13` `%r14` `%r15`
   - **返回地址**：`call` 压入的下一条指令地址，位于调用者与被调用者栈帧的边界处，供 `ret` 使用
   - **对齐**：栈帧的地址必须是 16 的倍数

     16 字节对齐，Attacklab 会用到哦

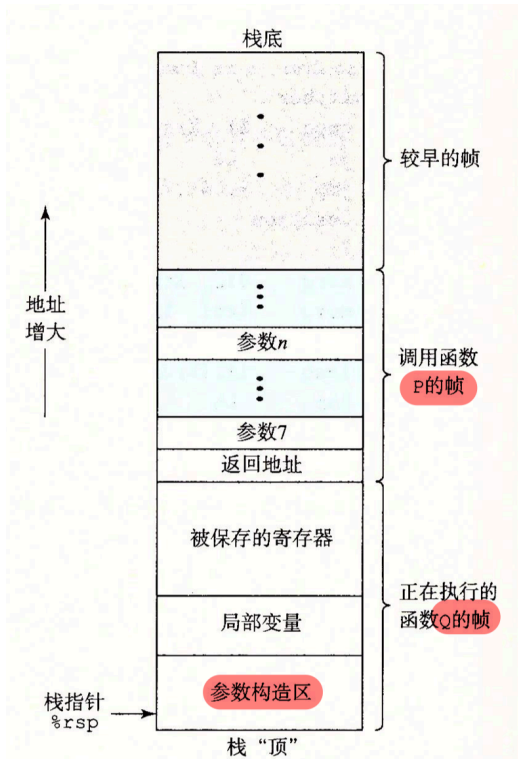


图 3-25 通用的栈帧结构(栈用来传递参数、存储返回信息、保存寄存器，以及局部存储。省略了不必要的部分)

x86-64 的栈结构

stack structure in x86-64

2. **栈顶管理**：使用 `push` 和 `pop` 指令进行数据的压栈和出栈管理。
3. **帧指针和栈指针**：使用寄存器 `%rbp` 和 `%rsp` 定位和管理栈帧。

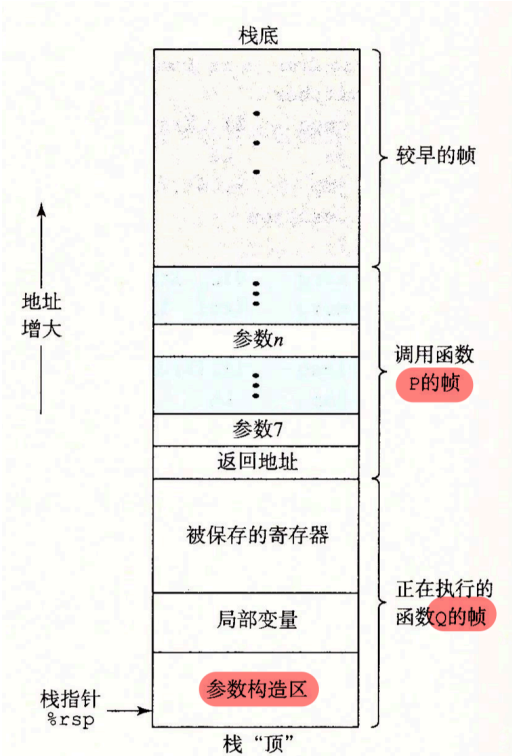


图 3-25 通用的栈帧结构(栈用来传递参数、存储返回信息、保存寄存器，以及局部存储。省略了不必要的部分)

转移控制

transfer control

转移控制是将程序的执行流程从一个函数跳转到另一个函数，并在完成任务后返回原函数。

- 指令层面，从函数 P 跳转到函数 Q，只需将程序计数器（PC）设置为 Q 的起始地址。
- 参数/数据层面，则需要通过栈帧 / 寄存器来传递。

call 和 ret 指令

`call` and `ret` instructions

在x86-64体系中，这个转移过程通过指令 `call Q` 和 `ret` 来完成：

- `call Q`：调用 Q 函数，并将返回地址压入栈，返回地址是 `call` 指令的下一条指令的地址。
- `ret`：从栈中弹出压入的返回地址，并将 PC 设置为该地址。

这样，程序可以在函数间跳转，并能够正确返回。

`call` 指令

- `call Label`： 直接调用，目标为标签地址
- `call *Operand`： 间接调用，目标为寄存器或内存中的地址
- 思考：什么时候会用到间接调用?
- 函数指针

`ret` 指令

- 执行返回，将返回地址从栈中弹出并跳转

call 和 ret 指令

`call` and `ret` instructions

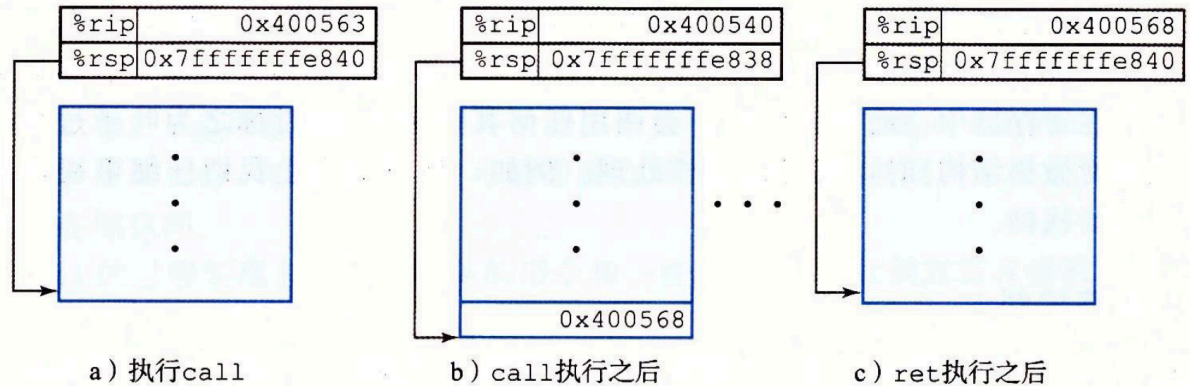


图 3-26 call 和 ret 函数的说明。call 指令将控制转移到一个函数的起始，而 ret 指令返回到这次调用后面的那条指令

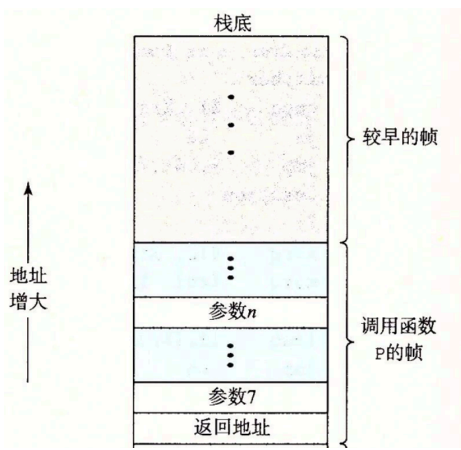
观察：

- 压栈后，`%rsp` -8，压入的是 `%rip` 下一条指令地址
- 弹栈后，`%rsp` +8，弹出的是栈帧中的内容，和当前运行时的 `%rip` 无关

数据传送

data transfer

- 前 6 个参数：通过寄存器传递
  - `%rdi` `%rsi` `%rdx` `%rcx` `%r8` `%r9`
- 剩余的参数：通过栈传递
  - 参数 7 在栈顶（低地址）
  - 参数构造区向 8 对齐



C 代码

```c
void proc(long a1, long *a1p, int a2, int *a2p,
short a3, short *a3p, char a4, char *a4p) {
    *a1p += a1;
    *a2p += a2;
    *a3p += a3;
    *a4p += a4;
}
```

生成的汇编代码

```asm
proc:
    movq 16(%rsp), %rax  # 取 a4p (64 位)
    addq %rdi, (%rsi)    # *a1p += a1 (64 位)
    addl %edx, (%rcx)    # *a2p += a2 (32 位)
    addw %r8w, (%r9)     # *a3p += a3 (16 位)
    movl 8(%rsp), %edx   # 取 a4 (8 位)
    addb %dl, (%rax)     # *a4p += a4 (8 位)
    ret
```

栈上的局部存储

local storage on the stack

有时，局部数据必须在内存中：

- 寄存器不够用
- 对一个局部变量使用地址运算符 `&`（因此必须能够为它产生一个地址，而不能放到寄存器里）
- 是数组或结构（要求连续、要求能够被引用 `&` 访问到）

注意，生长方向与参数构造区相反！

```
long call_proc() {
    long x1 = 1; int x2 = 2;
    short x3 = 3; char x4 = 4;
    proc(x1, &x1, x2, &x2, x3, &x3, x4, &x4);
    return (x1 + x2) * (x3 - x4);
}
```

```
    subq $32, %rsp          # 分配 32 字节的栈帧
    movq $1, 24(%rsp)       # 将 1 存储在 &x1
    movl $2, 20(%rsp)       # 将 2 存储在 &x2
    movw $3, 18(%rsp)       # 将 3 存储在 &x3
    movb $4, 17(%rsp)       # 将 4 存储在 &x4
    leaq 17(%rsp), %rax     # 创建 &x4
    movq %rax, 8(%rsp)      # 将 &x4 作为参数 8 存储
    movl $4, (%rsp)         # 将 4 作为参数 7 存储
    leaq 18(%rsp), %r9      # 将 &x3 作为参数 6
    movl $3, %r8d           # 将 3 作为参数 5
    leaq 20(%rsp), %rcx     # 将 &x2 作为参数 4
    movl $2, %edx           # 将 2 作为参数 3
    leaq 24(%rsp), %rsi     # 将 &x1 作为参数 2
    movl $1, %edi           # 将 1 作为参数 1

    # 调用 proc
    call proc

    # 从内存中检索更改
    movslq 20(%rsp), %rdx   # 获取 x2 并转换为 long
    addq 24(%rsp), %rdx     # 计算 x1 + x2
    movswl 18(%rsp), %eax   # 获取 x3 并转换为 int
    movsbl 17(%rsp), %ecx   # 获取 x4 并转换为 int
    subl %ecx, %eax         # 计算 x3 - x4
    cltq                    # 转换为 long
    imulq %rdx, %rax        # 计算 (x1 + x2) * (x3 - x4
    addq $32, %rsp          # 释放栈帧
```

寄存器上的局部存储

local storage on registers

PRO

被调用者 / 保存

CON

被 / 调用者保存

被调用者保存寄存器：`%rbx` `%rbp` `%r12` `%r13` `%r14` `%r15`

其他寄存器，**再除外** `%rsp`， 均为 “调用者保存” 寄存器

指针运算

pointer arithmetic

注意步长!

指针的加减会按照所指类型的大小缩放。数组名在大多数表达式中会转换为指向首元素的指针，因此可参与相同的指针运算。

> 如 `int*` 加减的步长是 `int` 的大小，即 4

同一个数组中的两个指针可以相减，结果等于两个地址之差 **除以元素大小**。

> 看最后一个示例

表达式	类型	值	汇编代码
`E`	`int*`	x_E	`movq %rdx, %rax`
`E[0]`	`int`	$M[x_E]$	`movl (%rdx), %eax`
`E[i]`	`int`	$M[x_E + 4i]$	`movl (%rdx, %rcx, 4), %eax`
`&E[2]`	`int*`	$x_E + 8$	`leaq 8(%rdx), %rax`
`E + i - 1`	`int*`	$x_E + 4i - 4$	`leaq -4(%rdx, %rcx, 4), %rax`
`*(E + i - 3)`	`int`	$M[x_E + 4i - 12]$	`movl -12(%rdx, %rcx, 4), %eax`
`&E[i] - E`	`long`	i	`movq %rcx, %rax`

数组分配和访问

array allocation and access

数组声明如：

```
T A[N];
```

其中 T 为数据类型，N 为整数常数。

初始化位置信息：

- 在内存中分配一个 $L \times N$ 字节的 **连续区域**，L 为数据类型 T 的大小（单位为字节）。
- 引入标识符 A，可以通过指针 x_A 访问数组元素。

访问公式：

$$\&A[i] = x_A + L \cdot i$$

数组名本身不是指针常量；它是数组对象的标识符，在大多数表达式中会转换为指向首元素的指针。

数组分配和访问

array allocation and access

如下声明的数组：

```
char A[12];
char B[8];
int C[6];
double D[5];
```

这些声明会生成带有以下参数的数组：

数组	元素大小	总的大小	起始地址	元素 `X[i]` 的地址
A	`char`：1	12	x_A	$x_A + i$
B	`char`：1	8	x_B	$x_B + i$
C	`int`：4	24	x_C	$x_C + 4i$
D	`double`：8	40	x_D	$x_D + 8i$

数组分配和访问

array allocation and access

假设 `E` 是一个 `int` 型数组，其地址存放在寄存器 `%rdx` 中，`i` 存放在寄存器 `%rcx`，那么 `E[i]` 的汇编代码为：

```
1   movl (%rdx, %rcx, 4), %eax # (Start, Index, Step)
```

特别地，对于数组下标 `A[i]` 的计算，实际上是 `*(A+i)`，即 `A+i` 是一个指针，指向 `A` 的第 `i` 个元素。

嵌套数组

```
1   T D[R][C]; // Row, Column
```

数组元素 `D[i][j]` 的内存地址为（解的时候顺序从左到右）：

$$\&D[i][j] = x_D + L \cdot (C \cdot i + j)$$

解码复杂指针表达式

decode complex expression

1. 从变量名开始
2. 往右读直到读到右括号或到底
3. 往左，忽略读过的
4. 上面的括号均不包括成对的括号
5. `*` 读作 “指针，指向”
6. `[x]` 读作 “长为 x 的数组，元素类型为”
7. `(…)` 读作 “函数，返回值为” + 上面提的成对括号（参数列表可选）
8. 如果为匿名类型，手动补充变量名（参数列表里会出现）

```
1   int *(*p[2])[3];
```

1. `p`：变量名
2. `[2]`：往右读，这是一个长度为 2 的数组，元素类型为…
3. `)`：往左读，忽略读过的
4. `(*p[2])`：这是一个指针，指向…
5. `(*p[2])[3]`：一个长度为 3 的数组
6. `;`：到底了，往左读
7. `*(*p[2])[3]`：一个指针，指向…
8. `int *(*p[2])[3]`：`int` 类型

合并：这是一个长度为 2 的数组，元素类型为指针，指向一个长度为 3 的数组，数组元素类型为指针，指向 `int`。

解码复杂表达式

decode complex expression

1. 从变量名开始
2. 往右读直到读到右括号或到底
3. 往左，忽略读过的
4. 上面的括号均不包括成对的括号
5. `*` 读作 “指针，指向”
6. `[x]` 读作 “长为 x 的数组，元素类型为”
7. `(…)` 读作 “函数，返回值为” + 上面提的成对括号（参数列表可选）
8. 如果为匿名类型，手动补充变量名（参数列表里会出现）

```
1   int func();
```

函数，返回值为 `int`

```
1   void func(int a, float b);
```

函数，返回值为 `void`，参数列表为 `int a` 和 `float b`

```
1   int* func();
```

函数，返回值为 `int*`

```
1   int (*func)();
```

函数指针，指向返回值为 `int` 的函数

解码复杂表达式

decode complex expression

Quiz

```
1   int *(*p[2])[3];
```

你可以在 cdecl.org 验证，或者尝试其他表达式。

让我们逐步解码：

1. `p`：变量名
2. `p[2]`：一个长度为 2 的数组，元素类型为…
3. `*p[2]`：一个指针，指向…
4. `(*p[2])[3]`：一个长度为 3 的数组，元素类型为…
5. `*(*p[2])[3]`：一个指针，指向…
6. `int *(*p[2])[3]`：`int` 类型

合并：声明 `p` 为一个包含 2 个元素的 数组 ，每个元素是 指向一个包含 3 个元素的 数组 的指针 ， 这些数组的元素是指向 `int` 类型的指针 。

数组名和指针

array name and pointer

数组名在大多数情况下的行为类似于指针。

相同

- 数组名在表达式中会被隐式转换为指向数组第一个元素的指针
- 可以对数组名和指针进行类似的指针算术操作

```
int value = *(arr + 2); // 等价于 arr[2]
```

相异

- 数组对象本身占据一块连续内存；指针变量只保存一个地址，可以指向数组元素、其他对象或动态分配的内存。

```
int arr[5]; // 局部数组，具有自动存储期
int *p = malloc(5 * sizeof(int)); // 动态分配的内存
```

- 数组名不是指针变量，也不是可修改左值，因此不能对它赋值；指针变量则可以重新赋值。

```
int arr[5];
int *p = arr;
p = NULL; // 合法，p 可以重新赋值
arr = NULL; // 非法，arr 不能重新赋值
```

数组名和指针

array name and pointer

数组名在大多数情况下的行为类似于指针。

相异

- 数组在声明时必须指定大小或初始化，而指针在声明时可以不初始化。

```c
int arr[5] = {1, 2, 3, 4, 5}; // 数组声明并初始化
int *p; // 指针声明，无需初始化
p = arr; // 之后初始化
```

- 对数组名使用 `sizeof` 操作符时，返回的是整个数组的大小，而对指针使用 `sizeof` 操作符时，返回的是指针本身的大小。

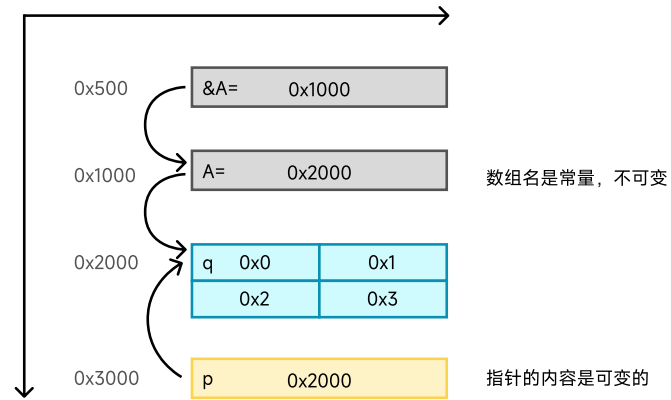
```c
int arr[5];
int *p = arr;
// %zu 代表 size_t 类型，通常用于表示 sizeof 操作符的结果
printf("%zu\n", sizeof(arr)); // 输出数组的总大小，20
printf("%zu\n", sizeof(p));   // 输出指针的大小，8
printf("%zu\n", sizeof(*p));  // 输出指针所指向的类型的大小，4
```

数组名和指针

array name and pointer

```
1   sizeof(q) = 4;
2   sizeof(A) = 16;
3   int* p = A; // 此处 A 转换为指向首元素的指针
4               // p 的值等于 &A[0]，但 p 与 A 的静态类型不同
5   sizeof(p) = 8; // sizeof 根据表达式的类型计算，因此结果不同
```

`A` 在 `sizeof` 中保持数组类型；赋值表达式中的 `A` 会转换为指向首元素的指针。



sizeof 与数组名

`sizeof` and array name

注意，当 `A` 是数组名，调用 `sizeof(A)` 时，返回的是整个数组的大小，如下所示：

```cpp
int main() {
    int A[5][3];
    cout << sizeof(&A) << endl; // 8: &A 的类型是 int (*)[5][3]，即指向整个数组的指针
    cout << sizeof(A) << endl; // 60: 在 sizeof 中 A 不转换为指针，类型仍为 int[5][3]
    cout << sizeof(*A) << endl; // 12: *A 的类型是 int[3]
    cout << sizeof(A[0]) << endl; // 12, A[0] 等价于 *A，即 int[3]
    cout << sizeof(**A) << endl; // 4: **A 是 int 类型的左值
    cout << sizeof(A[0][0]) << endl; // 4, A[0][0] 等价于 **A
    cout << &A << " " << (&A + 1) << endl; // &A + 1 跨过整个数组，即前进 60 字节
}
```

指针的大小与类型

pointer size and type

对于一个 `int* p`，`sizeof(p)` 等于?

答：`p` 的类型是 `int*`，所以 `sizeof(p)` 返回指针本身的大小 8；`sizeof(*p)` 返回 `int` 的大小 4。

辨析：`int q`

此时，`q` 的类型是 `int`，所以 `sizeof(q)` 等于 `sizeof(int)`，即 4。`q` 不是指针，不存在“其指向的内容”。

- `sizeof(expression)` 由表达式的类型决定
- 对指针 `p` 使用 `sizeof(p)` 得到指针类型的大小；使用 `sizeof(*p)` 得到所指类型的大小
- 数组表达式在 `sizeof`、一元 `&` 等少数上下文中不会转换为首元素指针

定长数组

fixed-length array

在处理定长数组时，编译器通过优化，可以尽可能避免开销较大的乘法运算。

原始的 C 代码

```c
int fix_prod_ele(fix_matrix A, fix_matrix B, long i, lo
    long j;
    int result = 0;
    for (j = 0; j < N; j++) {
        result += A[i][j] * B[j][k];
    }
    return result;
}
```

- 这是常规的固定矩阵乘法实现
- 迭代访问元素并计算乘积
- 因为使用了 `A[i][j]` 这种形式，所以每次访问一个矩阵元素时，都需要进行一次乘法运算。

优化的 C 代码

```c
int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i
    int *Aptr = &A[i][0];
    int *Bptr = &B[0][k];
    int *Bend = &B[N][k];
    int result = 0;

    do {
        result += *Aptr * *Bptr;
        Aptr++;
        Bptr += N;
    } while (Bptr != Bend);

    return result;
}
```

- 利用指针加速元素访问和乘法操作

变长数组

variable-length array

变长数组为灵活的数据存储解决方案。由于数组长度的不确定性，使用单个索引时容易导致性能问题。

初始 C 代码

```c
/* 计算变量矩阵乘积的函数 */
int var_prod_ele(long n, int A[n][n], int B[n][n], long
    long j;
    int result = 0;
    for (j = 0; j < n; j++) {
        result += A[i][j] * B[j][k];
    }
    return result;
}
```

- 由于使用了 `A[n][n]` 这种形式，而 `n` 是不能在编译时确定的变量，所以每次访问一个矩阵元素时，都需要进行一次乘法运算。
- 所以在访问变长数组元素时，被迫使用 `imulq` ，这可能会导致性能下降。

优化后的 C 代码

```c
/* 优化后的变量矩阵乘积计算函数 */
int var_prod_ele_opt(long n, int A[n][n], int B[n][n],
    int *Arow = A[i];
    int *Bptr = &B[0][k];
    int result = 0;
    long j;

    for (j = 0; j < n; j++) {
        result += Arow[j] * *Bptr;
        Bptr += n; // 向后移动指针，以减少访问时间
    }
    return result;
}
```

- 规律性的访问仍能被优化：通过定位数组指针，避免重复计算索引。

数据结构

data structure

`struct`

所有组分存放在内存中一段连续的区域内。

```
struct S3 {
    char c;
    int i[2];
    double v;
};
```

`union`

用不同的字段引用相同的内存块。

```
union U3 {
    char c;
    int i[2];
    double v;
};
```

类型	`c`	`i`	`v`	大小
S3	0	4	16	24
U3	0	0	0	8

可以看到，对于 `Union`，所有字段的偏移都是 0，因为它们共享同一块内存。

对齐

alignment

任何 `K` 字节的基本对象的地址必须是 `K` 的倍数。

K	类型
1	`char`
2	`short`
4	`int` `float`
8	`long` `double` `char*`

结构体对齐

structure alignment

在结构体的内存中，有一部分为了对齐而空出的字节

- 内部填充：使每个成员的偏移满足该成员类型的对齐要求
- 外部填充：使结构体总大小成为结构体对齐要求的整数倍，保证结构体数组中的每个元素都正确对齐

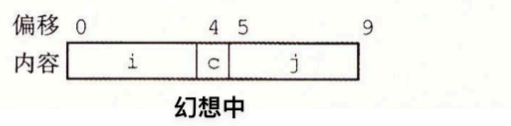
内部填充

internal padding

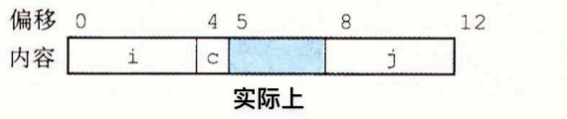
内部填充：为了让每个成员的起始地址满足该成员类型的对齐要求

```
struct S1 {
    int i;
    char c;
    int j;
};
```

虽然结构体的三个元素总共只占 9 字节，但为了满足变量 `j` 的对齐，内存布局要求填充一个 3 字节的间隙，这样 `j` 的偏移量将是 8，也就导致了整个结构体的大小达到 12 字节。



幻想中



实际上

外部填充

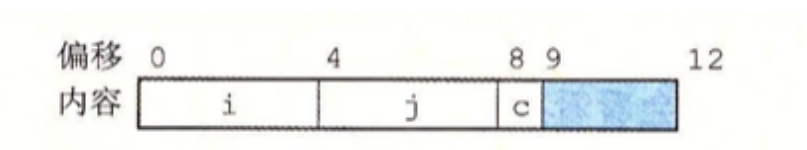
external padding

外部填充：为了让结构体总大小成为结构体对齐要求的整数倍

```
1   struct S2 {
2       int i;
3       int j;
4       char c;
5   };
```

此时，正常排确实可以只需要 9 字节，且同时满足了 `i` 和 `j` 的对齐要求。

但是，因为要考虑可能会有 `S2[N]` 这种数组声明，且数组又要求各元素在内存中连续，所以编译器实际上会为结构体分配 12 字节，最后 3 个字节是浪费的空间。



`.align 8` 命令可以确保数据的开始地址满足 8 的倍数。

结构体对齐示例

structure alignment example

定义

```
1   struct A {
2       char CC1[6];
3       int II1;
4       long LL1;
5       char CC2[10];
6       long LL2;
7       int II2;
8   };
```

回答以下问题：

1. `sizeof(A)` 为？6(2) + 4(4) + 8 + 10(6) + 8 + 4(4) = 56
2. 若将结构体重排，尽量减少结构体的大小，得到的新结构体大小？
   6 + 10 + 4 + 4 + 8 + 8= 40

技巧：

- 尽量减少结构体的大小：通常将对齐要求较大的成员放在前面，并把对齐要求相同的成员放在一起
- 结构体的对齐取其所有成员对齐要求的最大值；嵌套的 `union`、`struct` 使用其类型自身的对齐要求

结构体对齐示例

structure alignment example

定义

```c
typedef union {
    char c[7];
    short h;
} union_e;

typedef struct {
    char d[3];  // 4
    union_e u;  // 8
    int i;      // 4
} struct_e;

struct_e s;
```

回答以下问题：

1. `s.u.c` 的首地址相对于 `s` 的首地址的偏移量是？4
2. `sizeof(union_e)` 为？8
3. `s.i` 的首地址相对于 `s` 的首地址的偏移量是？12
4. `sizeof(struct_e)` 为？16
5. 若只将 `i` 的类型改成 `short`，那么 `sizeof(struct_e)` 为？14
6. 若只将 `h` 的类型改成 `int`，那么 `sizeof(union_e)` 为？8
7. 若将 `i` 的类型改成 `short`，将 `h` 的类型改成 `int`，那么 `sizeof(union_e)` 为？`sizeof(struct_e)` 为？8 16
8. 若将 `short h` 的定义删除，那么 (1)~(4) 间的答案分别是？3 7 12 16

强制对齐

force alignment

- 任何内存分配函数（`alloca` `malloc` `calloc` `realloc`）生成的块的起始地址都必须是 16 的倍数
- 大多数函数的栈帧的边界都必须是 16 字节的倍数（这个要求有一些例外）
- 参数构造区向 8 对齐



Thank you for your listening!

2026/10/1

© SHANGHAI HENIAN INFORMATION TECHNOLOGY CO.,LTD